

# Generate–Verify–Repair for Intent-Driven Network Changes: Controlled Evaluation with a Deterministic Verifier

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We evaluate whether a generate–verify–repair (GVR) pipeline—single-shot LLM generation followed by a deterministic network verifier and up to one automated repair—reduces accepted unsafe network changes versus LLM-only generation on a standardized synthetic NetOps intent workload. In a preregistered paired design (study-hosted-netops-gvr-v1-2026-08-29) we ran  $N=500$  distinct synthetic intents (shared single initial candidate per intent) with generation by gpt-5-mini and adjudication by a deterministic verifier. Results: guarded (GVR) accepted 500/500 final changes; baseline (LLM-only) accepted 496/500 (paired difference = 0.008). The preregistered primary exact conditional McNemar two-sided test on the discordant pairs ( $n_{10} = 4$ ,  $n_{01} = 0$ ) yields  $p = 0.125$  (computed as  $p = 2 * \text{PBinom}(X \geq 4; n = 4, p = 0.5)$ ). An exploratory paired bootstrap percentile 95% CI (10,000 resamples, seed = 1729) = [0.002, 0.016] (bootstrap labeled exploratory per preregistration and interpreted cautiously given only four discordants). Repairs were invoked for 4 initial candidates; total model calls used = 504 (500 shared\_initial + 4 repair calls). The preregistered templated-automation comparator was not executed. Because the preregistered decision rule is the exact conditional McNemar ( $p = 0.125$ ), we do not claim confirmatory statistical significance. We reconcile the conditional McNemar and the exploratory bootstrap CI, document verifier scope and known blind spots, provide execution accounting and representative validator traces, and give explicit artifact pointers and hashes for reproducibility.

## I. INTRODUCTION

Operator intent→device configuration translation can reduce manual effort, but errors in generated CLI sequences or transient unsafe states can cause outages. Modern large language models (LLMs) show promise for translating human intent to device configuration, yet hallucination and factual errors remain central reliability concerns for operational NetOps. A practical mitigation architecture is generate–verify–repair (GVR): produce a candidate configuration with an LLM, deterministically verify it against encoded workflow and policy rules, and, if verification fails, run a bounded automated repair attempt. This paper reports a preregistered, artifact-grounded paired experiment that asks: Does a GVR pipeline (LLM → deterministic verifier → up to one automated repair) produce fewer accepted unsafe changes than LLM-only generation on a standardized synthetic NetOps intent workload? We contribute: (1) a preregistered paired trial ( $N=500$  paired intents) executed end-to-end with archived artifacts; (2) an artifact-grounded description of generation, verifier semantics, and the bounded repair loop; (3) reconciled confirmatory inference (two-sided conditional exact McNemar) and ex-

ploratory paired bootstrap CI descriptions with an explicit small-discordant-count caveat; and (4) execution accounting, representative validator traces, and a discussion of verifier blind spots and practical external-validity limits.

## II. RELATED WORK

Three converging literatures inform the reliability challenge for LLM-driven NetOps and motivate a GVR architecture. First, survey and systems work documents active interest in using LLMs for intent translation, configuration generation, AIOps, and multi-agent orchestration, while emphasizing hallucination and factual errors as deployment barriers. Large surveys and system descriptions synthesize these trends and identify practical obstacles such as vendor heterogeneity, control-plane timing, and dataset availability [doi:10.1145/3718958.3750537; <https://openalex.org/W4411735779>]. Intent-driven configuration prototypes and frameworks further demonstrate feasibility of LLM-based translation but typically pair those generators with ad-hoc validation or operator oversight rather than large, controlled trials against deterministic verifiers [doi:10.1002/nem.2313]. These works establish that translation quality alone is insufficient for operational acceptance and motivate tool-augmented pipelines that embed validation as a gate.

Second, methodological and tool-integration work advocates for explicit generate-validate-repair (or RAG/tool-augmented) evaluation practices and standardized validator semantics. Recent methodological papers propose blueprints for evaluating retrieval- or tool-augmented LLMs in operational contexts and call for measuring verifier coverage, failure modes, and orchestration costs rather than only generator fidelity [<https://openalex.org/W4403443637>]. These studies highlight three evaluation gaps that our experiment targets: (a) need for end-to-end trial designs that preserve pairing between conditions on identical intents; (b) the value of frozen preregistration to avoid post-hoc analysis choices; and (c) the importance of quantifying model-call overhead per avoided unsafe change when verifiers are invoked.

Third, deterministic configuration verifiers and scalable verification systems provide concrete oracles for many syntactic and workflow properties but exhibit known modeling limits. Work on scalable, distributed verifiers (Batfish derivatives and localized subspecification approaches)

demonstrates that automated checks can detect availability/sequence and policy violations at scale, yet these verifiers vary in which control-plane or vendor semantics they model and therefore differ in the set of hazards they can detect [https://openalex.org/W4413757123]. Prior verifier papers typically measure verifier expressiveness and scalability; they do not, however, couple that verifier deterministically with an LLM generator in a preregistered paired trial that then measures repaired vs accepted unsafe changes across a broad task bank. Our study uses a `deterministic_netops_validator_v5` that implements canonical parsing, required-sequence/workflow checks, availability/group constraints, and paired-field dependency rules—i.e., the class of properties many verifier systems aim to cover—but we acknowledge, as prior work does, that verifier pass does not imply absence of vendor-specific or timing hazards.

Across these literatures, two important empirical gaps remain. First, although several system papers and frameworks argue for verifier-assisted or agentic LLM pipelines, there are few publicly archived, preregistered paired comparisons that measure how bounded repair loops change the rate of verifier-detectable unsafe outputs across a large, stratified set of intents; most prior evaluations are descriptive, single-condition, or lack frozen analysis plans [https://openalex.org/W4413757586]. Second, methodological reviews emphasize that evaluation must report verifier codebooks, failure-mode counts, and orchestration costs (model calls, repair rates) to allow operational judgment [https://openalex.org/W4403443637]; without those details, claims about reliability gains remain qualitative. Our experiment addresses both gaps by preregistering a paired confirmatory estimand, archiving the verifier codebook and representative validator traces, reporting provider audit counts (model\_calls\_used = 504; repair = 4), and publishing the paired contingency table for reproducible exact-test calculation. In short, we position our work as an artifact-grounded trial that operationalizes the recommendations from recent methodological work and connects generator performance to deterministic verifier coverage in a controlled paired design.

### III. METHODOLOGY

**Preregistration and confirmatory plan:** The experimental design, sampling, estimands, and primary decision rule were preregistered as study-hosted-netops-gvr-v1-2026-08-29. The primary estimand is the paired success-rate difference (GVR - baseline) across  $N = 500$  distinct intents stratified evenly over 10 synthetic topology profiles (50 intents per profile). Sampling seeds, the deterministic task generator, and the analysis executor were frozen prior to execution (preregistration manifest SHA-256 = aa8982a27c73e51521ac023a78adcf358ef3978c0f9bc05213801fcbd2f1d0a). The preregistered confirmatory decision rule is the two-sided conditional exact McNemar computed on the observed discordant pairs; the preregistration also specified an exploratory paired bootstrap percentile interval (10,000 resamples; bootstrap\_seed = 1729) as a descriptive interval only.

**Task generation and paired design:** A deterministic task generator (`deterministic_netops_task_generator_v5`) produced 500 distinct intent tasks (task\_manifest entries archived). Each intent was executed under two paired conditions using the same shared\_initial\_generation candidate: (1) baseline—accept the shared initial LLM candidate as the final change; and (2) guarded (GVR)—validate the shared candidate with the deterministic verifier and, if failing, invoke at most one automated repair attempt. Pairing ensured within-intent control of task difficulty, topology profile, and the single shared initial candidate design as preregistered.

**Generation model and orchestration:** The declared generation model for the run is gpt-5-mini (declared\_model\_version recorded in execution/model\_configuration.json; execution/model\_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). Per the execution manifest and provider audit, the run produced 500 shared\_initial\_generation calls and 4 repair calls (hosted\_model\_call\_event\_count = 504; model\_calls\_used = 504). The orchestration adapter family is hosted\_netops\_gvr\_v1 using the hosted provider recorded as 'openai\_responses'. The maximum\_repair\_calls\_per\_task was set to 1 in the preregistration and enforced by the controller.

**Deterministic verifier: role, semantics, and outputs:** The primary adjudicator is `deterministic_netops_validator_v5` (validator\_version = 5.0), implemented as a deterministic canonicalization and rule-checking pipeline. For each candidate CLI sequence, the verifier executes the following stages in order: (1) syntactic parsing and canonical normalization (tokenization of interface commands, canonical ordering of multi-line sequences, removal of formatting variants); (2) command-level required-sequence enforcement (patterns such as must\_be\_down\_for\_fields, all\_down\_before\_any\_field\_change, make\_before\_break switchover); (3) availability/group constraints (exactly\_active, minimum\_active\_in\_groups); (4) dependency and paired-field constraints (paired MTU changes, equal\_final\_fields between interfaces); and (5) final-state comparison against the task's expected\_final\_state. Any nonempty violation set yields a failing outcome. For each episode the verifier emits a structured validator\_trace containing fields including normalized\_configuration, observed\_touched\_field\_count, parsed\_command\_count, required\_command\_count, violation\_count and list, validation\_before/after summaries, and boolean valid. Representative verifier\_trace objects are archived under execution/scoring/ (see representative artifact examples). Passing the verifier is a necessary experimental safety gate but is not claimed to be sufficient for all real-world hazards (see Threats to Validity).

**Verifier codebook and artifact pointer:** The human-readable verifier codebook (violation-code → short description) used to construct repair prompts and to interpret validator\_trace violations is archived as an artifact referenced by the execution manifest. The execution bundle includes a full hash manifest; the execution manifest path is execution/execution\_manifest.json (the manifest enumerates the ver-

ifier codebook artifact path and its SHA-256 for direct retrieval). Representative scoring JSON objects in `execution/scoring/` contain the violation codes emitted for each failing candidate.

Repair policy, prompt template, and acceptance rule: When the verifier returns `valid=false` for a shared initial candidate under the guarded condition, the GVR controller issues at most one repair prompt to the same generation model. The repair prompt is constrained and intentionally compact: it contains (a) the original high-level intent abstraction for the task, (b) the verifier violation codes and short descriptions extracted from the `verifier_trace`, and (c) a request for a corrected canonical CLI sequence that satisfies the listed violation codes. The repair template minimizes additional contextual material to isolate the marginal effect of one repair attempt. A repair candidate is accepted only if a subsequent verifier invocation returns `valid=true`. All repair prompts, responses, and verifier outcomes are logged and archived in `execution/provider_calls/` and `execution/scoring/`.

Scoring rule and outcome definition: Per preregistration, each intent’s final accepted change is scored binary by the primary verifier: `success = final accepted configuration passes deterministic_netops_validator_v5 (valid=true)` versus `unsafe = final accepted configuration fails (valid=false)`. For baseline pairs the shared candidate is the accepted change without verification-based repair; for guarded pairs acceptance required verifier pass after repair (if any). The analysis used complete pairs only (`complete_pair_count = 500`) per the preregistered missingness plan; technical failures are recorded and were zero in this run (`failed_episode_count = 0`).

Statistical procedures and reproducible calculations: The preregistered primary test is the two-sided conditional exact McNemar operating on discordant pairs. Let  $n_{10}$  be the number of pairs where guarded succeeds and baseline fails, and  $n_{01}$  the reverse;  $n = n_{10} + n_{01}$  and  $k = n_{10}$ . The conditional exact two-sided p-value is computed as  $p = 2 * \text{PBinom}(X \geq k; n, 0.5)$ , with clipping at 1.0. In this run the archived paired contingency (`analysis/paired_contingency_table.csv`; archive hash listed in the evidence bundle) is  $n_{11} = 496$ ,  $n_{10} = 4$ ,  $n_{01} = 0$ ,  $n_{00} = 0$ , so  $n = 4$  and  $k = 4$ . For  $n = 4$ ,  $\text{PBinom}(X \geq 4; n = 4, p = 0.5) = (1/16)$ , giving  $p = 2*(1/16) = 0.125$ . The exact McNemar implementation and this conditional binomial formula were preregistered and applied exactly as documented here; the contingency CSV and analysis logs are archived for reproducibility.

Exploratory paired bootstrap: The preregistration additionally specified an exploratory paired bootstrap percentile CI for descriptive interpretation. We resampled paired outcomes with replacement (10,000 resamples; `bootstrap_seed = 1729`) and computed the paired success-rate difference (guarded - baseline) per resample; the 2.5th and 97.5th percentiles produce the reported exploratory interval  $[0.002, 0.016]$ . The bootstrap was explicitly labeled exploratory in the preregistration because percentile intervals can have poor frequentist coverage when discordant counts are very small and sample margins are near ceiling; we therefore present the bootstrap as a descriptive

interval and rely on the preregistered exact McNemar for the primary confirmatory decision.

Provider accounting, determinism, and retry policy: Provider audit fields and the deterministic reconciliation artifact record `hosted_model_call_event_count = 504` (`completed_hosted_model_call_event_count = 504`; `failed_hosted_model_call_event_count = 0`), `stage_counts = {shared_initial_generation: 500, repair: 4}`, and `model_calls_used = 504`. The controller enforced `maximum_attempts_per_call = 1` and `maximum_repair_calls_per_task = 1` as preregistered. All random seeds used for task generation, bootstrap resampling, and other nondeterministic steps are recorded in the archived artifacts to ensure computational reproducibility.

Predefined exclusion, missingness, and sensitivity: Per preregistration there were no exclusions applied; `complete_pair_count = 500` and no technical failures occurred. The preregistration specified a sensitivity analysis treating technical failures as unsafe; because no such failures occurred the primary and sensitivity treatments coincide. All missingness, contamination, and reconciliation artifacts are archived (`analysis/missingness_summary.csv`, `analysis/contamination_summary.csv`, `deterministic_reconciliation.json`).

Reproducibility artifacts and pointers: Key archived artifacts referenced in this methodology include `execution/raw_results.jsonl` (SHA-256 = `fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02`), `execution/task_manifest.jsonl` (SHA-256 = `5a822e787302a0b3bb03a86a81b20564a44e2636731f853f9d45ac12e4876cc8`), `analysis/paired_contingency_table.csv` (SHA-256 = `8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece`), and `analysis/analysis_log.jsonl` (SHA-256 = `261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e`). The full execution manifest (`execution/execution_manifest.json`) contains the complete list of artifact paths and SHA-256 digests for direct retrieval. Reproducible calculation parameters (bootstrap resamples, `bootstrap_seed = 1729`) and the exact McNemar formula are recorded in `analysis/analysis_log.jsonl`.

Implementation notes and limits applied to the methodology: The verifier enforces canonical, task-specified properties and exact final-state equality against the synthetic `expected_final_state`; it intentionally omits vendor-specific parser idiosyncrasies, control-plane timing/simulation, and out-of-band hardware constraints. The repair policy is intentionally bounded (single repair) to limit model-call overhead and to measure the marginal corrective value of one repair attempt in an operationally conservative budget. These design choices were preregistered and are explicitly documented to bound claims about operational generality.

#### IV. RESULTS

Execution and primary counts: The run completed `completed_episode_count = 1000` ( $500 \text{ intents} \times 2 \text{ conditions}$ ) with `model_calls_used = 504`. `analysis/condition_summary.csv`

(archived hash: f3e197425cc03dec41cda77f078f6d0b079b06bbde7ac736de5fe988b027ac58) reports baseline successes 496/500 (0.992) and guarded successes 500/500 (1.000). The paired contingency table (analysis/paired\_contingency\_table.csv; archived hash: 8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece) is:  $n_{11} = 496$ ,  $n_{10} = 4$ ,  $n_{01} = 0$ ,  $n_{00} = 0$ . Repairs were invoked for four initial candidates; all four repair attempts resulted in subsequent verifier passes and correspond to the four discordant pairs ( $n_{10} = 4$ ). Mean model calls per accepted change  $\approx 504/500 = 1.008$  and median calls per accepted change = 1 (most accepted changes required only the shared\_initial\_generation). Provider audit (deterministic\_reconciliation.json in the evidence bundle) records hosted\_model\_call\_event\_count = 504, completed\_hosted\_model\_call\_event\_count = 504, failed\_hosted\_model\_call\_event\_count = 0, cache\_hit\_count = 0, cache\_miss\_count = 504, and stage\_counts: shared\_initial\_generation = 500, repair = 4.

Confirmatory exact McNemar calculation (preregistered primary decision rule): Observed discordants  $n = n_{10} + n_{01} = 4$  with  $k = n_{10} = 4$ . Using the conditional exact binomial formulation described in Methods,  $p = 2 * \text{PBinom}(X \geq 4; n = 4, p = 0.5)$ . For  $n = 4$ ,  $\text{PBinom}(X \geq 4; n = 4, p = 0.5) = 1/16$ , so  $p = 2 * (1/16) = 0.125$ . Because this exact conditional test was the preregistered decision rule, we do not claim confirmatory statistical significance for the primary estimand.

Exploratory paired bootstrap interval and its interpretation: The preregistered exploratory paired bootstrap percentile 95% CI (10,000 resamples; seed = 1729; artifacts archived as analysis/paired\_difference\_figure.svg and analysis/analysis\_log.jsonl; analysis\_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e) for the paired difference is [0.002, 0.016]. This bootstrap is explicitly exploratory in the preregistration. Practically, with near-ceiling margins (496 and 500 successes) and only four discordant pairs, the conditional exact McNemar conditions on the observed margins and treats discordants as a small binomial sample; under that conditioning the two-sided tail probability yields  $p = 0.125$ . The bootstrap resamples paired outcomes without conditioning on margins and can therefore produce a nonzero lower CI bound even when the conditional McNemar is non-significant. Small discordant counts impair the frequentist coverage properties of percentile bootstrap intervals; we therefore present the bootstrap interval as a descriptive, exploratory summary rather than as a second confirmatory decision. This distinction follows the preregistered analysis\_plan and multiplicity handling.

Repair diagnostics and representative artifacts: The four repaired episodes were flagged by the verifier for verifier-detectable sequencing or availability violations (e.g., missing make\_before\_break availability ordering or required must\_be\_down\_for\_fields violations). Each repaired episode received a single constrained repair prompt that included the intent abstraction and the verifier

violation codes; the repair candidate accepted by the verifier corrected the flagged sequence or availability issue and returned valid=true. Representative verifier\_trace JSON entries and their fields (validation\_before, validation\_after, violation\_count, normalized\_configuration) are archived under execution/scoring/ for inspection. Example representative scoring JSONs (archived artifacts included in the evidence bundle) include: - execution/scoring/task-000001-baseline.json (SHA-256 = e97212b62733ba30de344b8980d06b59cfecb2d791daa91332b3d617a52f45f8) — shows validator\_version = 5.0 and a completed valid=true outcome for the shared candidate; the file includes validation\_before/after blocks and the normalized\_configuration. - execution/scoring/task-000002-baseline.json (SHA-256 = 2d458e3e083065215264fa509e0df3fed38cf160881f7e6a143c4ddd83034b92) — covers a 'make\_before\_break\_uplink\_switchover' hard pattern. These representative artifacts illustrate the verifier emitted fields and normalization semantics; full per-episode raw results and scoring JSONs are listed in the execution manifest and raw\_results.jsonl (execution/raw\_results.jsonl SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02).

Contingency accounting and provider audit consistency: The deterministic\_reconciliation.json produced by the analysis executor confirms accounting consistency: task\_count = 500 (paired intents), planned\_episode\_count = 1000, completed\_episode\_count = 1000, complete\_pair\_count = 500, baseline\_success\_count = 496, guarded\_success\_count = 500, and contingency entries  $n_{11} = 496$ ,  $n_{10} = 4$ ,  $n_{01} = 0$ ,  $n_{00} = 0$ . Provider\_call\_audit in the reconciliation artifact matches the execution manifest stage\_counts and the model\_calls\_used summary. These machine-verifiable reconciliation artifacts and their hashes are archived in the evidence bundle for independent reproduction of the contingency table and the McNemar calculation.

Operational interpretation and cost implication: In this synthetic workload the bounded GVR controller intercepted four verifier-detectable hazards at low LLM overhead (repair rate 0.8%, four additional model calls). Mean model-call overhead per accepted change was  $\sim 0.008$  extra calls beyond the single shared initial generation. These observations show that a compact verifier plus a bounded repair loop can remove verifier-detectable unsafe outputs cheaply in this controlled setting; scaling to production requires measuring verifier runtime, expanding verifier fidelity (vendor parsing, timing/convergence models), and stress testing with adversarial NetOps inputs where repair frequency and cost may increase.

Synthesis relative to preregistered power assumptions: The preregistered power plan targeted an absolute paired increase  $\approx 0.08$ . The observed baseline success rate ( $496/500 = 0.992$ ) left little headroom and produced only four discordant pairs, substantially reducing realized confirmatory sensitivity relative to that plan. This ceiling effect explains why an empirically small paired difference (0.008) yields an exploratory bootstrap CI excluding zero while the preregistered conditional exact

McNemar is non-significant; both results are reported and interpreted according to the preregistered analysis\_plan.

## V. DISCUSSION

Interpretation and reconciliation of inferential outcomes: The GVR pipeline prevented four verifier-detectable unsafe initial candidates that baseline would have accepted. Under the preregistered conditional exact McNemar ( $p = 0.125$ ) this difference is not confirmatory. The exploratory bootstrap CI excludes zero but should be interpreted cautiously: the conditional McNemar conditions on observed margins and evaluates the binomial tail of discordant outcomes; with  $n = 4$  and  $k = 4$  this yields  $p = 0.125$ . The bootstrap resamples pairs without fixing margins and can produce a nonzero lower CI bound when most pairs are concordant and a small number are discordant. Because the bootstrap was predeclared exploratory and because small discordant counts reduce frequentist coverage guarantees for percentile intervals, we report both inferential perspectives and follow the preregistered decision rule for the primary claim.

Verifier coverage and concrete blind spots: The deterministic verifier enforces canonical normalization, required sequencing, availability/group constraints, dependency and paired-field rules, and exact final-state equality to the task expected\_final\_state. It does not model vendor-specific CLI semantics, timing/convergence effects in control-plane protocols, vendor idiosyncrasies, hardware resource constraints, or out-of-band side effects. Passing this verifier is necessary for experiment safety but not sufficient for all production hazards; expanding verifier fidelity (vendor parsers, control-plane simulators, timing models) is required before operational automation. The human-readable verifier codebook (violation-code  $\rightarrow$  description) is enumerated in the execution manifest (see execution/execution\_manifest.json in the evidence bundle) and the manifest lists the exact codebook artifact path and its SHA-256 for direct retrieval.

Operational implications and cost tradeoffs: A compact deterministic verifier plus a bounded repair loop intercepted verifier-detectable hazards at low LLM overhead in this synthetic workload. Observed repair rate was 4/500 (0.8%), implying modest additional model calls under the bounded policy. Production adoption requires (a) measuring verifier runtime and scalability on real inventories, (b) extending verifier fidelity to vendor/timing effects, and (c) evaluating adversarial inputs where repair frequency and cost may increase. The provider audit (hosted\_model\_call\_event\_count = 504) quantifies the empirical model-call overhead for this run.

Threats to validity and generality: Internal validity is strong because of preregistration, fixed seeds, and complete pairs (complete\_pair\_count = 500; execution\_manifest\_failed\_episode\_count = 0). Construct validity is limited by verifier abstractions and synthetic device templates (synthetic\_topologies\_v1 and synthetic\_device\_configs\_v1). External validity is constrained by single-model scope (gpt-5-mini) and synthetic topologies; multi-model replication and real vendor CLI tests are required to generalize. Conclusion

validity was affected by realized margins: the preregistered power plan targeted an absolute paired increase  $\approx 0.08$ ; the observed near-ceiling baseline (0.992) produced only four discordant pairs and reduced realized power relative to the preregistered assumption. These limitations are documented in the archived manifest and preregistration.

## VI. LIMITATIONS

- Synthetic tasks and topology profiles were used (synthetic\_topologies\_v1 and synthetic\_device\_configs\_v1); external validity to production hardware, vendor-specific CLIs, live telemetry, and control-plane timing effects is untested.
- Only a single generation model (gpt-5-mini) was evaluated; multi-model generality and replication remain to be established.
- Safety in this experiment is defined relative to deterministic\_netops\_validator\_v5's encoded rule set (syntactic parsing/normalization, required-sequence/workflow-policy checks, protected-interface rules, group and availability constraints, dependency-rule enforcement, and final-state comparison to the task expected\_final\_state). Passing this verifier is necessary for the experiment's outcome but not sufficient for all real-world safety properties.
- The preregistered power plan targeted an absolute paired increase  $\approx 0.08$  (8 percentage points). The observed baseline success rate ( $496/500 = 0.992$ ) produced only four discordant pairs, substantially reducing realized power relative to the preregistered assumption and limiting confirmatory sensitivity.
- The preregistered templated-automation comparator (secondary confirmatory arm) was not executed; therefore the preregistered multiplicity plan (two confirmatory tests with Bonferroni correction) could not be fully applied and the familywise error interpretation is partial.

## VII. CONCLUSION

In a preregistered paired trial ( $N = 500$ ) using gpt-5-mini and a deterministic verifier, a generate-verify-repair pipeline repaired four verifier-detectable unsafe initial candidates at modest model-call cost (4 repairs; model\_calls\_used = 504). The paired difference = 0.008 yields an exploratory bootstrap 95% CI [0.002, 0.016], while the preregistered exact conditional McNemar test ( $n_{10} = 4, n_{01} = 0$ ) yields  $p = 0.125$ ; under the preregistered decision rule we do not claim confirmatory significance. Deterministic verification plus a bounded repair loop can remove verifier-detectable unsafe outputs with low overhead in synthetic settings; broader operational claims require expanded verifier fidelity, adversarial NetOps evaluation, multi-model replication, and execution of the preregistered templated comparator (which was not executed in this run). All primary execution and analysis artifacts cited here are archived in the evidence bundle for independent verification; see the archived execution manifest for full artifact paths and hashes.

## DISCLOSURE STATEMENT

This study was executed autonomously after an explicit autonomy lock. Generation model used in the archived run: gpt-5-mini (declared\_model\_version recorded in execution/model\_configuration.json; execution/model\_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). Orchestration adapter family: hosted\_netops\_gvr\_v1 using the hosted provider recorded as 'openai\_responses'. Key automated components recorded in the run: deterministic\_netops\_task\_generator\_v5 (task synthesis), deterministic\_netops\_validator\_v5 (primary deterministic verifier/scorer), and the GVR controller implementing shared\_initial\_generation plus an automated single-repair step (maximum\_repair\_calls\_per\_task = 1). Preregistration: study-hosted-netops-gvr-v1-2026-08-29 (preregistration SHA-256 = aa8982a27c73e51521ac023a78adcfa358ef3978c0f9bc05213801fcbd2f1d0a). The immutable initial master prompt is archived at provenance/master\_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Primary high-level execution quantities reported here (N = 500 complete pairs; model\_calls\_used = 504; repair calls = 4; paired counts n11 = 496, n10 = 4, n01 = 0, n00 = 0) are derived from archived execution and analysis artifacts. Representative artifact hashes included in the evidence bundle: execution/raw\_results.jsonl SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02; execution/task\_manifest.jsonl SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f9d45ac12e4876cc8; analysis/paired\_contingency\_table.csv SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece; analysis/analysis\_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e. The human-readable verifier codebook (violation-code → description) and the full list of artifact paths and hashes are enumerated in the archived execution manifest (execution/execution\_manifest.json); the execution manifest lists the exact verifier codebook artifact path and its SHA-256 for direct retrieval. The design, preregistration, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revision were performed autonomously after the autonomy lock; no human manually edited or rewrote manuscript technical content after that lock. The preregistered templated-automation comparator was not executed in this archived run; that unexecuted arm means the originally preregistered two-test Bonferroni familywise plan could not be fully applied.

## REFERENCES

- [1] doi:10.1145/3718958.3750537
- [2] Sebastian Simon, Alina Mailach, Johannes Dorn, Norbert Siegmund, "A Methodology for Evaluating RAG Systems: A Case Study On Configuration Dependency Validation", 2024, 10.48550/arxiv.2410.08801
- [3] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [4] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
- [5] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537